

Lab 1: Requirements Engineering & UML Use-Case Modelling

Problem Statement #44 — Database Query Performance Profiler

Domain: Developer Tools & IT Operations | Actors: Database Administrator, Backend Lead

System overview: A database observability tool that ingests slow query logs, parses SQL execution plans, identifies missing index candidates, and generates weekly performance optimization digests for database administrators and backend engineering leads.

1. Requirements Table

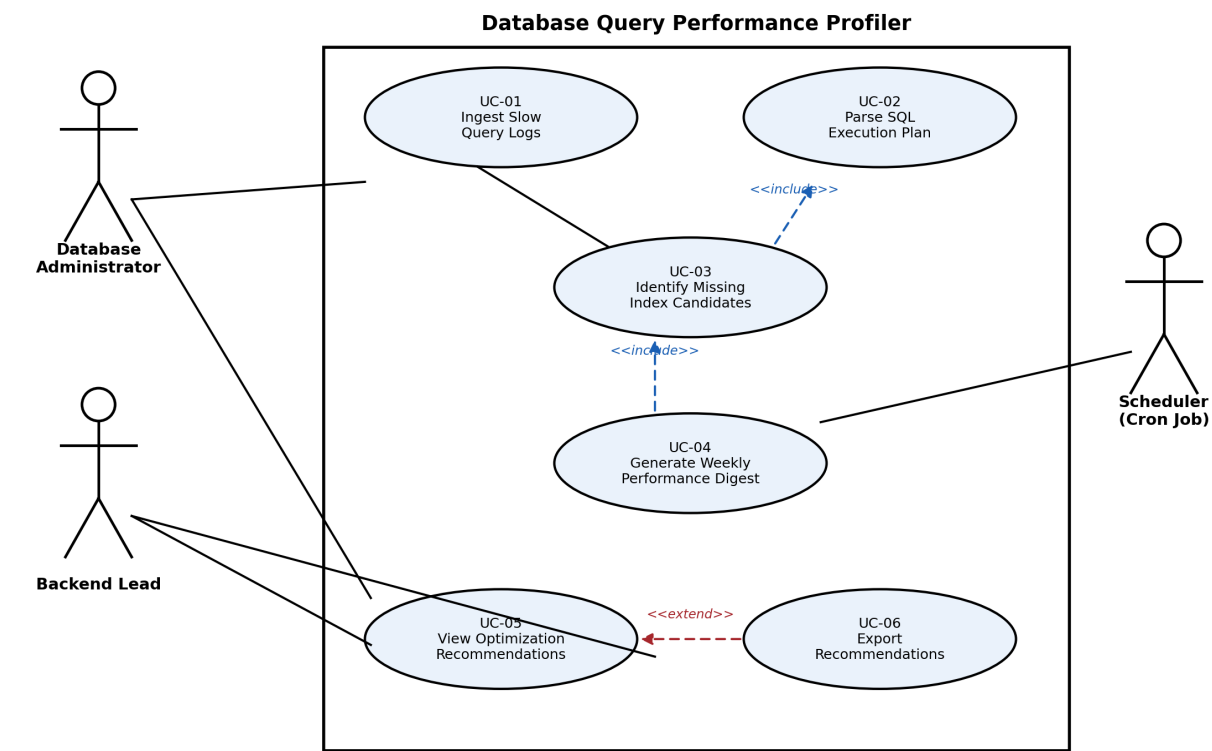
Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall parse Postgres/MySQL EXPLAIN plans, highlight sequential table scans on large tables, and recommend optimal index column definitions.	High	Pass: Missing composite index identified on filtered columns. Fail: Query plan with sequential scan marked optimized.	Core value proposition of the profiler; without accurate plan parsing no downstream recommendation is trustworthy.
FR-002	Functional	The system shall ingest slow query logs from Postgres and MySQL instances (via pg_stat_statements and the MySQL slow query log), normalizing each entry into a common schema (timestamp, query text, duration, source host).	High	Pass: A log entry from either engine appears in the normalized log store within 30 seconds of being written. Fail: Entries are dropped or fields are missing after ingestion.	A single normalized pipeline is required before any parsing, analysis, or digesting can occur.
FR-003	Functional	The system shall generate a weekly performance optimization digest summarizing the top 10 slowest queries, all missing-index recommendations, and estimated latency improvement per recommendation.	High	Pass: Digest is generated every Monday 06:00 UTC and lists exactly the top 10 queries by cumulative duration. Fail: Digest is missing, incomplete, or generated on the wrong schedule.	Automates recurring manual review work and keeps the Backend Lead informed without requiring them to query logs directly.
FR-004	Functional	The system shall allow a Database Administrator to configure a query-duration alert threshold and notify the Backend Lead by email or Slack when a query exceeds it.	Medium	Pass: A query exceeding the configured threshold triggers a notification within 2 minutes. Fail: No notification is sent, or it is sent for a query under the threshold.	Enables proactive response to regressions rather than waiting for the weekly digest.
FR-005	Functional	The system shall allow a Database Administrator or Backend Lead to export index recommendations as CSV or ready-to-run SQL DDL (CREATE INDEX) scripts.	Medium	Pass: Exported DDL executes without syntax error against the source schema. Fail: Exported file is malformed or omits recommended columns.	Reduces the manual effort of translating a recommendation into an executable schema change.
NFR-001	Performance & Security	Slow query log ingestion must handle up to 5,000 log records per minute without dropping entries or slowing the host database.	High	Pass: Benchmarking tests confirm target latency and security standards under simulated peak load. Fail: Ingestion drops records or measurably increases host DB latency at 5,000 records/min.	Protects production database performance, which is the very thing the tool exists to safeguard.

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
NFR-002	Reliability & Availability	The recommendation dashboard shall be available 99.5% of the time per calendar month, and the weekly digest job shall complete within 10 minutes of its scheduled trigger time.	Medium	Pass: Uptime monitoring shows greater than or equal to 99.5% availability over a rolling 30-day window and digest completion logs show completion under 10 minutes. Fail: Availability drops below 99.5% or digest generation exceeds 10 minutes.	The DBA and Backend Lead depend on timely, available reporting to act on optimization opportunities before they compound.

Note: FR-001 and NFR-001 reproduce the sample requirements supplied in the problem statement; FR-002–FR-005 and NFR-002 were newly authored for this system.

2. UML Use-Case Diagram

Actors: **Database Administrator** and **Backend Lead** (primary human stakeholders, per the problem statement) and **Scheduler (Cron Job)**, a secondary system actor that triggers the weekly digest automatically. The diagram includes one **<<include>>** relationship (UC-04 always includes UC-03, and UC-03 always includes UC-02) and one **<<extend>>** relationship (UC-06 optionally extends UC-05).



ID	Use Case
UC-01	Ingest Slow Query Logs
UC-02	Parse SQL Execution Plan
UC-03	Identify Missing Index Candidates («include» UC-02)
UC-04	Generate Weekly Performance Digest («include» UC-03)
UC-05	View Optimization Recommendations
UC-06	Export Recommendations («extend» UC-05)

3. Use-Case Flow Specification

Use Case: UC-03 — Identify Missing Index Candidates

Actors	Database Administrator (primary); Scheduler (Cron Job) may also trigger this use case indirectly via UC-04.
Trigger	A newly parsed SQL execution plan (UC-02) is available for analysis, or the Database Administrator manually requests analysis of a specific query.
Preconditions	(1) At least one slow query log entry has been ingested (UC-01). (2) The corresponding EXPLAIN plan has been successfully parsed (UC-02). (3) The database schema metadata (existing indexes, table statistics) is accessible to the system.
Postconditions	Success: One or more index-candidate recommendations are persisted and linked to the source query, each with an estimated improvement score, ready for display (UC-05) or export (UC-06). Failure: No recommendation is created and the query is flagged as "analysis incomplete" for retry.

Main Success Scenario

- 1 The system retrieves the parsed execution plan for the query (produced by UC-02, «include»).
- 2 The system scans the plan for sequential (full table) scans on tables above the configured row-count threshold.
- 3 For each flagged sequential scan, the system inspects the filter/WHERE and JOIN columns used by the query.
- 4 The system cross-references those columns against the existing index metadata for the affected table.
- 5 The system determines that no existing index (or composite index) covers the filtered columns.
- 6 The system computes an estimated latency improvement by comparing the current scan cost to the projected cost with a candidate index.
- 7 The system generates a recommended index definition (columns and order) and a confidence score.
- 8 The system persists the recommendation and links it to the source query for later viewing (UC-05) or export (UC-06).
- 9 Use case ends successfully.

Alternate Flow

5a. Existing Index Already Covers the Query

- 5a1. The system finds that an existing index (or composite index) already covers the filtered columns.
- 5a2. The system marks the query as "optimized" rather than generating a new recommendation.
- 5a3. The system logs the query as reviewed with zero missing-index candidates, and the use case resumes at step 9 (ends successfully) with no recommendation created.